KI-basierte Angriffserkennung in kritischen Infrastrukturen

Basim Bennaji
Telematics - Computer Science Department
Freie Universität Berlin
Berlin, Germany
basim.bennaji@fu-berlin.de

***Zusammenfassung*—Die fortschreitende Digitalisierung kritischer Infrastrukturen (KRITIS) in den Sektoren Energie, Verkehr und Gesundheit birgt neben enormen Effizienzgewinnen eine stark vergrößerte Angriffsfläche für Cyberbedrohungen. Da traditionelle, signaturbasierte Schutzmechanismen bei neuartigen Angriffen (sogenannten Zero-Day-Exploits) an ihre Grenzen stoßen, rückt der Einsatz von Künstlicher Intelligenz (KI) zur anomaliebasierten Angriffserkennung zunehmend in den Fokus. Die vorliegende Arbeit evaluiert im Rahmen eines „Proof of Concept“ die Leistungsfähigkeit und praktische Einsetzbarkeit von fünf Machine-Learning-Algorithmen – Decision Tree, Random Forest, XGBoost, Logistische Regression und einem Neuronalen Netz – im Kontext von Intrusion Detection Systemen (IDS). Als Datengrundlage dient der umfangreiche CSE- CIC-IDS2018-Datensatz, der modernes Netzwerkverhalten und aktuelle Angriffsszenarien realitätsnah abbildet. Die experimentellen Ergebnisse demonstrieren, dass baumbasierte Algorithmen (insbesondere Decision Tree und XGBoost) komplexeren Deep-Learning-Ansätzen bei der Klassifikation von strukturierten, tabellarischen Netzwerkdaten deutlich überlegen sind. XGBoost und der Decision Tree klassifizierten über 1,5 Millionen Testdaten nahezu fehlerfrei bei sehr kurzen Trainingszeiten von unter einer Minute. Das Neuronale Netz erzielte zwar ebenfalls eine hohe Erkennungsrate, produzierte jedoch signifikant mehr Fehlalarme, was im regulären Betrieb zu Alert Fatigue (Alarmmüdigkeit) der Administratoren führen kann. Die Logistische Regression scheiterte an den nicht-linearen Mustern der Netzwerkdaten. Die Arbeit schließt mit der Erkenntnis, dass ressourceneffiziente Ensemble-Modelle für den Einsatz in kritischen Infrastrukturen derzeit die verlässlichste Lösung darstellen.**

I. Einleitung

A. Motivation und Ausgangslage

Das heutige Leben ist ohne kritische Infrastruktur (KRITIS) wie in den Bereichen Stromversorgung, Gesundheit und Verkehr kaum möglich. Die meisten Menschen schenken dieser Infrastruktur im Alltag kaum Beachtung, da sie, wenn sie reibungslos funktioniert, im Hintergrund bleibt. Deshalb ist es zwingend notwendig, dass diese Infrastruktur immer einwandfrei funktioniert. Selbst ein kleiner Ausfall dieser Systeme kann gravierende Folgen nach sich ziehen, unter anderem auch für Menschenleben. Aber auch schwere wirtschaftliche Verluste können daraus folgen. Kritische Infrastruktur ist also vor allem für die Sicherheit des öffentlichen Lebens von großer Bedeutung. Daher muss sie auch besonders geschützt werden.

In den vergangenen Jahren wurden verstärkt Systeme der kritischen Infrastruktur modernisiert. Durch die Digitalisierung und Vernetzung wird eine effizientere Arbeitsweise angestrebt, was dazu beitragen soll, mögliche Ausfälle durch ineffiziente Prozesse zu reduzieren und die Ausfallsicherheit zu erhöhen. Grundsätzlich ist dieser technologische Fortschritt sehr erwünscht, da Betreiber ihre operativen Kosten signifikant senken können. Jedoch bringt diese Entwicklung neue Gefahren mit sich.

Die zunehmende Vernetzung ermöglicht eine Vielzahl neuartiger Angriffsvektoren. Insbesondere werden hierdurch Fernangriffe („Remote-Angriffe“) möglich. Angreifende Akteure, auch aus Drittländern, müssen nicht mehr physisch vor Ort sein, sondern können aus großer Entfernung die kritische Infrastruktur manipulieren und somit die Versorgungssicherheit gefährden. Daher gilt es genau abzuwägen, in welchem Verhältnis der Nutzen der Digitalisierung zur vergrößerten Angriffsfläche steht.

Das bedeutet auch, dass Angreifer zunehmend professioneller werden und über erhebliche Ressourcen verfügen. Zu den Angreifern zählen heutzutage organisierte kriminelle Hackergruppen, wirtschaftliche Gegner oder auch Drittstaaten. Dabei können sie von Spionage bis hin zu Sabotage alles betreiben.

Da diese Angriffe stetig komplexer und anspruchsvoller werden, ist es wichtiger denn je, diese Systeme proaktiv zu schützen und sicher zu halten. Hier sind Intrusion Detection Systems ein zentraler technologischer Baustein, um die Netzwerke kontinuierlich auf Angriffe oder verdächtige Aktivitäten zu überwachen und Administratoren bei Verdacht rechtzeitig zu alarmieren.

B. Problemstellung und regulatorischer Rahmen

Herkömmliche Angriffserkennung basiert auf traditionellen, signaturbasierten Methoden. Diese signaturbasierten „*Intrusion Detection Systems*“ beobachten den Netzwerkverkehr und suchen nach bekannten Mustern und ermöglichen eine sehr schnelle und zuverlässige Erkennung bereits dokumentierter Angriffe. Sie sind jedoch konzeptionell ungeeignet, um völlig neuartige, unbekannte Angriffe (sogenannte Zero-Day-Exploits) zu erkennen, da für diese noch keine Signaturen existieren [1].

Das ist auch der Grund, weshalb anomaliebasierte Intrusion Detection Systems immer mehr an Bedeutung gewinnen. Denn bei diesen Systemen wird ein so genannter "*Normalzustand*“ für das Netzwerkverhalten definiert. Jede Abweichung von

diesem *Normalzustand* wird als potenzieller Angriff erkannt und gemeldet. Dadurch wird es möglich, auch bisher unbekannte Angriffsvektoren zu identifizieren.

Die Notwendigkeit solcher *Intrusion Detection Systems* für Systeme kritischer Infrastrukturen zur Erkennung von Angriffen, auch unbekannten, spiegelt sich auch in der aktuellen Gesetzgebung wider. Mit dem IT-Sicherheitsgesetz 2.0 hat der Gesetzgeber die Anforderungen an kritische Infrastrukturen verschärft und den Einsatz von Systemen zur Angriffserkennung (SzA) gemäß § 8a Abs. 1a BSIG seit dem 1. Mai 2023 verbindlich vorgeschrieben. Das Bundesamt für Sicherheit in der Informationstechnik (BSI) fordert hierbei explizit eine kontinuierliche Überwachung und Auswertung relevanter Betriebsparameter. Da die traditionellen signaturbasierten Verfahren konzeptionell nicht in der Lage sind, die geforderte Erkennung unbekannter Bedrohungen zu gewährleisten, ist der Einsatz von anomaliebasierten Erkennungssystemen zur Erfüllung der gesetzlichen Mindestanforderungen so gut wie unumgänglich geworden [2]. Dies ist insbesondere vor dem Hintergrund der Nachweispflicht gegenüber dem BSI von Bedeutung, da Unternehmen die wirksame Umsetzung dieser Maßnahmen alle zwei Jahre belegen müssen.

Obwohl die Anomalieerkennung nicht zwingend mit Hilfe von Machine-Learning-Verfahren realisiert werden muss, bieten sich diese dennoch an, um die großen Datenmengen effizient zu verarbeiten.

C. Zielsetzung und Forschungsfragen

Das Hauptziel dieser Arbeit besteht in der Entwicklung und Evaluierung verschiedener Machine-Learning-Modelle zur Anomalieerkennung im Sinne eines „*Proof of Concept*“. Dabei soll ermittelt werden, wie effektiv künstliche Intelligenz Muster in Netzwerkdaten analysieren kann, um bösartigen Netzwerkverkehr von harmlosem zu unterscheiden. Der Fokus der Arbeit liegt dabei ausdrücklich auf der technischen Machbarkeit und dem systematischen Vergleich der Algorithmen, nicht auf der Entwicklung eines final einsatzbereiten Softwareprodukts.

Im Rahmen dieser Arbeit sollen folgende Fragen beantwortet werden:

- Wie gut funktionieren Machine-Learning-Modelle generell bei der Erkennung von Cyberangriffen auf Kommunikationssysteme?
- Welche Art von Machine-Learning-Modell ist für diese spezifische Aufgabe am besten geeignet?
- Ist der Einsatz solcher Modelle sinnvoll unter Einbeziehung von Faktoren wie Falsch-Positiv-Raten, Latenz und Skalierbarkeit?

Hierzu werden fünf unterschiedliche Architekturansätze verglichen: *Decision Tree*, *Random Forest*, *XGBoost*, *Logistische Regression* und *Neuronales Netz*.

Um eine realitätsnahe Evaluation zu gewährleisten, wird der öffentlich zugängliche Datensatz CSE-CIC-IDS2018 von der University Of New Brunswick verwendet [3]. Dieser Datensatz bildet eine moderne Netzwerkinfrastruktur sowie sieben verschiedene Angriffsszenarien ab und dient als Grundlage für das Training und die Bewertung der Modelle.

D. Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in sechs Kapitel wie folgt: Im hierauf folgenden Kapitel 2 werden die informationstechnischen Grundlagen von Intrusion Detection Systemen näher beleuchtet und die theoretischen Funktionsweisen der ausgewählten Machine-Learning-Algorithmen vorgestellt. Kapitel 3 beschäftigt sich mit dem experimentellen Aufbau. Hier werden der verwendete Datensatz, die Datenvorverarbeitung (Preprocessing) sowie die Implementierung der Modelle detailliert beschrieben. In Kapitel 4 werden die Ergebnisse der trainierten Algorithmen anhand von Performance-Metriken präsentiert und miteinander verglichen. Kapitel 5 diskutiert diese Ergebnisse kritisch im Hinblick auf ihre praktische Einsetzbarkeit in kritischen Infrastrukturen. Die Arbeit schließt in Kapitel 6 mit einem zusammenfassenden Fazit und beantwortet die im ersten Kapitel gestellten Fragen.

E. Verwandte Arbeiten

Die Anwendung von Machine-Learning-Verfahren zur netzwerkbasierten Anomalieerkennung, insbesondere auf Basis des CSE-CIC-IDS2018-Datensatzes, ist Gegenstand zahlreicher aktueller Forschungsarbeiten. So evaluierten beispielsweise Wicaksana *et al.* [4] die Leistung baumbasierter Ensemble-Methoden und bestätigten die technische Überlegenheit von XGBoost gegenüber klassischen Random-Forest-Modellen, primär hinsichtlich der reinen Klassifikationsgenauigkeit und Vorhersagegeschwindigkeit.

Während viele dieser bestehenden Arbeiten primär die reine Klassifikationsleistung in Laborumgebungen optimieren, fokussiert sich diese Arbeit auf die Anforderungen in kritischen Infrastrukturen. Neben der generellen Erkennungsrate sind hierbei die Reduktion von Fehlalarmen (False Positives) zur Vermeidung von *Alert Fatigue* [5] sowie die Optimierung der Trainingslatenzen durch VRAM-Speichermanagement wichtig, um einen praxistauglichen Einsatz in produktiven Echtzeitumgebungen zu evaluieren.

Für weiterführende theoretische Grundlagen und alternative Ansätze zur netzwerkbasierten Anomalieerkennung sei an dieser Stelle auf die Arbeiten in [6] und [7] verwiesen.

II. Intrusion Detection System (IDS)

Ein *Intrusion Detection System* ist ein sicherheitstechnisches System, das Computernetzwerke kontinuierlich analysiert und nach böswilligen Angriffen oder anderen verdächtigen Aktivitäten Ausschau hält. Sobald das System einen potenziellen Angriff, eine Bedrohung oder eine Abweichung von der Norm registriert, generiert es einen Alarm und meldet diesen an die Administratoren des Netzwerks.

Hierbei geht es nur um das Erkennen und nicht um das Verhindern oder Lösen von Angriffen. Das ist ein wichtiger Unterschied zwischen IDS und *Intrusion Prevention Systemen* (IPS). Diese beiden Systeme muss man gut abgrenzen können, da sie zwei verschiedene Aufgaben haben. Ein IPS greift aktiv

in das Netzwerk ein und versucht, erkannte Angriffe durch das Blockieren von Datenpaketen eigenständig zu verhindern. Häufig werden beide Systeme, IDS und IPS, gemeinsam verwendet. Das IDS erkennt die Angriffe und das IPS versucht diese zu stoppen (siehe Fig. 1).

In der Praxis werden IDS häufig so aufgebaut, sodass der Netzwerkverkehr gespiegelt wird. Auf dem gespiegelten Verkehr arbeitet das IDS, indem es diesen analysiert. Das hilft Latenzen oder Betriebsunterbrechungen gering zu halten. Beispielsweise werden *False-Positives*, also normaler Netzwerkverkehr der als Angriff erkannt wird, erst von einem Admin überprüft werden, bevor das IPS das System womöglich lahmlegt. Ebenfalls ist es in diesem Kontext wichtig Latenzen gering zu halten, da es sich bei diesen Systemen oft um zeitkritische Aufgaben handelt.

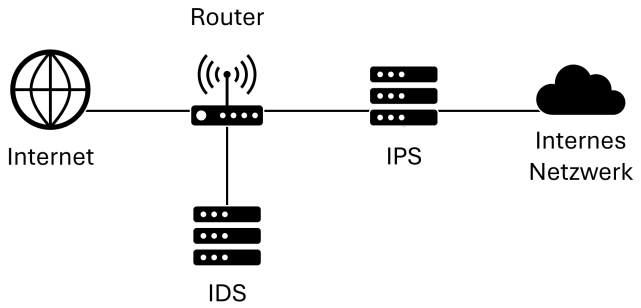

Abbildung 1. Architektonische Gegenüberstellung von Intrusion Detection und Intrusion Prevention Systemen (Eigene Darstellung in Anlehnung an [8]).

A. Arten von IDS

Je nach Betrachtungsweise lassen sich Intrusion Detection Systeme in verschiedene Kategorien unterteilen. Hinsichtlich ihrer Architektur und ihres Einsatzortes unterscheidet man primär fünf verschiedene Ansätze [8] [9]:

- **Netzwerkbasiertes Intrusion-Detection-System (NIDS):** Diese Architektur bildet den Schwerpunkt dieser Arbeit. Ein NIDS überwacht ein gesamtes Netzwerk, indem es den Verkehr zwischen verschiedenen Punkten passiv analysiert.
- **Hostbasiertes Intrusion-Detection-System (HIDS):** Ein HIDS wird meist auf einem Computer installiert. Dieses überwacht Systemlogs, Datenintegrität und Aktivitäten. Es fokussiert sich auf das eine Gerät auf dem es installiert ist,
- **Protokollbasiertes Intrusion-Detection-System (PIDS):** Ein PIDS überwacht und analysiert für ein bestimmtes Protokoll den Verkehr. Dabei wird es meist auf Webservern installiert, um die Kommunikation zwischen Host und Client zu überwachen.
- **Anwendungsprotokollbasiertes Intrusion-Detection-System (APIDS):** APIDS überwachen applikationsspezifische Protokolle. Darunter zählen auch SQL-Transaktionen zwischen Client und Host.
- **Hybrides Intrusion-Detection-System:** Bei hybriden Intrusion Detection Systemen werden mehrere der zuvor erklärten Ansätze verwendet, um Angriffe auf verschiedenen Ebenen besser erkennen zu können.

Unabhängig von der gewählten Architektur und dem Einsatzort lassen sich all diese Systeme hinsichtlich ihrer zugrundeliegenden Erkennungsmethode in zwei primäre Kategorien unterteilen: signaturbasierte und anomaliebasierte IDS.

B. Erkennungsmethoden: Signatur- vs. Anomaliebasiert

Wie in Fig. 2 und Fig. 3 schematisch dargestellt, unterscheiden sich moderne IDS primär in zwei Ansätzen: der signaturbasierten und der anomaliebasierten Erkennung.

Signaturbasierte Erkennung Bei der signaturbasierten Erkennung hält das System dauerhaft Ausschau nach Mustern und Signaturen von bereits bekannten Angriffen. Dies geschieht häufig durch Methoden wie Zeichenerkennung („Pattern Matching“) oder den Abgleich von spezifischen Verhaltensmustern mit einer bestehenden Datenbank. Der große Vorteil dieser Methode liegt in ihrer hohen Geschwindigkeit und Zuverlässigkeit: Bekannte Bedrohungen werden fast in Echtzeit erkannt, und das System ist durch den simplen Aufbau und die Funktionsweise einfach zu verstehen. Das ist hilfreich für die Administratoren, da sie hierdurch genau verstehen, weshalb das IDS bei einem bestimmten Datenfluss einen Alarm ausgelöst hat. Dadurch können False-Positives schnell erkannt werden. Wobei es bei dieser Kategorie von IDS ohnehin eine sehr geringe False-Positive-Rate für bekannte Angriffe gibt.

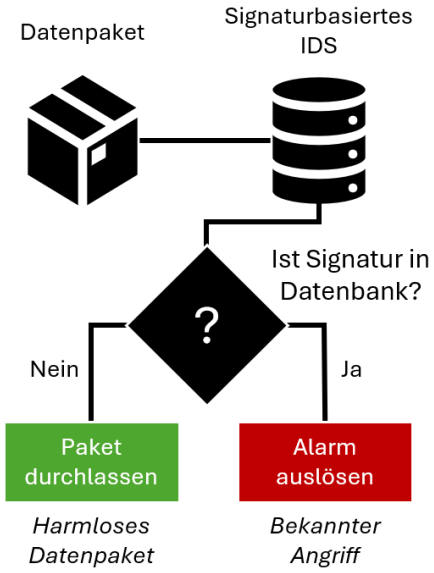

Abbildung 2. Ablauf der signaturbasierten Erkennung mittels Abgleich in einer Angriffsdatenbank (Eigene Darstellung).

Signaturbasierte IDS werden in Netzwerken häufig eingesetzt, um den äußeren Ring eines Netzwerkes vor Angriffen

zu schützen. Der entscheidende Nachteil ist jedoch, dass neue, dem System noch unbekannte Angriffe (Zero-Day-Exploits [1]) prinzipbedingt nicht erkannt werden können. Zero-Day-Vulnerabilities sind Sicherheitslücken, die von den Entwicklern nicht entdeckt wurden, jedoch von externen Akteuren, mit einem Exploit, nutzbar gemacht wurden. Werden diese Sicherheitslücken den Entwicklern nicht gemeldet, bleiben diese oft lange ungelöst. Dadurch, dass diese Lücken nicht öffentlich bekannt sind, gibt es auch oft keine Signatur. Das ist der Grund, weshalb signaturbasierte IDS immer aktuell gehalten werden müssen, um ein hohes Sicherheitsniveau zu gewähren.

Anomaliebasierte Erkennung Bei der anomaliebasierten Erkennung wird für ein System ein „Normalzustand“ (Baseline) für den regulären Netzwerkverkehr definiert. Dieser Normalzustand wird in der Regel ermittelt, indem das System den historischen, fehlerfreien Netzwerkverkehr über einen gewissen Zeitraum analysiert und statistische Durchschnittswerte lernt, etwa für das genutzte Datenvolumen, typische Port-Zugriffe oder die durchschnittliche Paketgröße. Alle signifikanten Änderungen, beispielsweise Ereignisse oder Datenflüsse, die signifikant hiervon abweichen, werden als Anomalie klassifiziert und gemeldet.

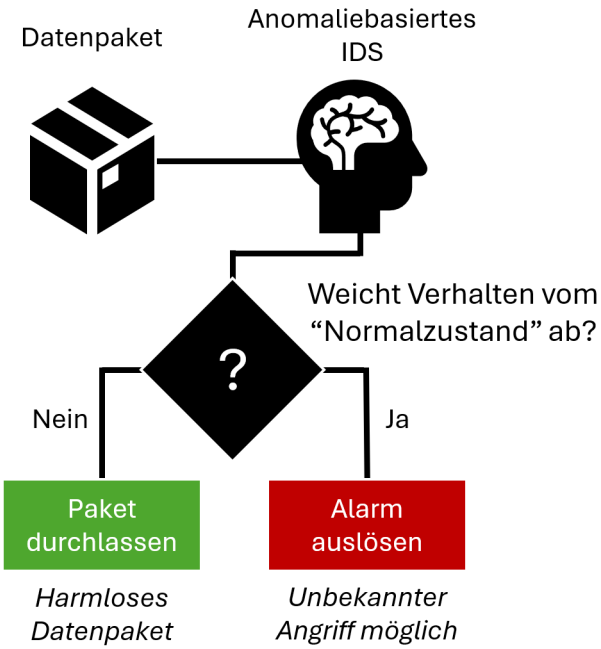

Abbildung 3. Ablauf der anomaliebasierten Erkennung mittels Klassifikation durch Machine Learning (Eigene Darstellung).

Dadurch ist es bei dieser Kategorie von IDS möglich, neue, unbekannte Angriffe zu erkennen. Der Nachteil hierbei ist jedoch, dass die False-Positive-Rate deutlich höher ist als bei signaturbasierten IDS, was eine manuelle Überprüfung der Meldungen unerlässlich macht. Vor allem, wenn sich das legitime Verhalten von Nutzern oder Systemen ändert, können viele Fehlalarme entstehen.

Anomaliebasierte IDS werden meist im Inneren eines Netzwerks eingesetzt, da hier oft abweichendes Nutzerverhalten einen Angriff bedeutet. Wenn sich ein Angreifer von Innen Zugang verschafft, kann dieser potenziell große Datenmengen extrahieren, um diese später zu verkaufen. Dabei fließt eine Menge Daten über einen Nutzer, was vom IDS als Anomalie erkannt werden kann.

C. Machine-Learning-Modelle zur Anomalieerkennung

In dieser Arbeit werden fünf verbreitete Machine-Learning-Algorithmen daraufhin untersucht, wie gut sie Netzwerkanomalien klassifizieren können. Die Algorithmen unterscheiden sich grundlegend in ihrer Komplexität und ihrer Funktionsweise.

*1) **Decision Tree**:* Der Decision Tree ist ein vergleichsweise einfaches, intuitives Modell, das den Datensatz anhand von logischen Ja/Nein-Regeln aufteilt und nach klaren Trennlinien in den Daten sucht. Ein wesentlicher Vorteil dieses Modells ist, dass es sehr schnell trainiert werden kann und die getroffenen Entscheidungen für den Menschen transparent und gut überprüfbar sind. Eine große Schwäche des „Decision Trees“ liegt jedoch darin, dass dieser dazu neigt, die Trainingsdaten äuswendig zu lernen"(Overfitting), wodurch sie bei neuen, unbekannten Daten unter Umständen an Präzision verlieren.

*2) **Random Forest**:* Der Random Forest baut auf dem Prinzip des Decision Trees auf, nutzt jedoch die Methode des sogenannten Ensemble-Learnings [10]. Anstatt eines einzelnen Baumes werden hunderte Entscheidungsbäume trainiert, die jeweils nur eine zufällige Teilmenge der Daten betrachten. Die finale Klassifikation (Angriff oder nicht) erfolgt durch eine Mehrheitsentscheidung aller Bäume. Dies macht das Modell äußerst robust gegen Ausreißer und verringert das Risiko des Overfittings signifikant. Der Nachteil ist jedoch der Verlust der Transparenz; das Modell wird zu einer "Black Box“ und ist für Menschen weitaus weniger nachvollziehbar als ein einzelner Entscheidungsbaum.

*3) **eXtreme Gradient Boosting (XGBoost)**:* Wie der Random Forest gehört auch XGBoost zur Familie der baumbasierten Ensemble-Modelle [10], nutzt jedoch ein anderes Lernprinzip: das sogenannte Boosting-Verfahren. Anstatt hunderte Bäume gleichzeitig und unabhängig voneinander zu trainieren, werden die Entscheidungsbäume bei XGBoost sequenziell (nacheinander) aufgebaut. Dabei wird jeder neue Baum gezielt darauf trainiert, die Fehler seiner Vorgänger zu korrigieren. Dieses Vorgehen macht den Algorithmus sehr leistungsstark und präzise, weshalb er insbesondere bei strukturierten, tabellarischen Daten, wie Netzwerk-Logs, oft verwendet wird. Zudem ist XGBoost stark auf Recheneffizienz und Geschwindigkeit optimiert. Ähnlich wie beim Random Forest geht jedoch auch hier die direkte Nachvollziehbarkeit der Entscheidungsfindung im Vergleich zu einem einzelnen, simplen Decision Tree verloren [11].

*4) **Logistische Regression**:* Die Logistische Regression ist ein klassisches statistisches Verfahren, welches versucht, eine mathematische Trennlinie (Hyperebene) zwischen harmlosem Traffic und Angriffen zu finden. Obwohl sie schnell und

effizient bei einfachen, linear trennbaren Problemen ist, stößt sie bei IDS-Daten oft an ihre Grenzen. Netzwerkangriffe weisen in der Regel hochkomplexe und nicht-lineare Muster auf, weshalb eine einfache lineare Trennung häufig zu hohen Fehlerraten führt.

*5) **Neuronales Netz:*** Neuronale Netze, wie das in dieser Arbeit verwendete Multi-Layer Perceptron (MLP), sind klassische KI-Modelle, deren Struktur von biologischen Gehirnen inspiriert ist und aus mehreren Schichten verbundener Knoten (Neuronen) besteht. Sie sind in der Lage, extrem komplexe und nicht-lineare Zusammenhänge in Daten zu erkennen, die für lineare Modelle oder den Menschen unsichtbar bleiben. Diese Leistungsfähigkeit hat auch einen großen Nachteil: Neuronale Netze benötigen weitaus mehr Rechenleistung, oft in Form von Grafikprozessoren (GPUs), und große Datenmengen. Zudem zeigt die Praxis oft, dass sie bei rein tabellarischen Datensätzen (wie Log-Dateien) nicht zwingend bessere Ergebnisse liefern als baumbasierte Modelle wie der Random Forest.

III. Methodik und Experimenteller Aufbau

Dieses Kapitel detailliert den praktischen Aufbau des Experiments. Es umfasst die Beschreibung des verwendeten Datensatzes, die Schritte der Datenvorverarbeitung (Preprocessing) zur Vorbereitung für das maschinelle Lernen sowie die Implementierungsdetails und die gewählte Software-Architektur.

A. Verwendeter Datensatz

Für das Training und die Evaluation der Machine-Learning-Modelle wird der öffentlich zugängliche Datensatz CSE-CIC-IDS2018 verwendet [3]. Dieser Datensatz wurde von der University of New Brunswick (UNB) in Kanada in Zusammenarbeit mit dem Communications Security Establishment (CSE) generiert. Dieser gilt als sehr umfangreich und modern für die Evaluation von Intrusion Detection Systemen, da eine aktuelle Netzwerktopologie sowie reale Angriffsszenarien dargestellt werden. Es werden zwei Organisationen simuliert: Eine angreifende Organisation, sowie eine angegriffene Organisation. Das Angreifer Netzwerk besteht aus 50 Computern, die sowohl Microsoft Windows als auch Linux Betriebssysteme verwenden. Das angegriffene Netzwerk ist deutlich größer und in fünf Abteilungen geteilt. Die Abteilungen beinhalten insgesamt 420 Computer, auf denen hauptsächlich Microsoft Windows 8.1/10 als Betriebssystem installiert ist, wobei 30 Computer Ubuntu Linux haben. Zu den fünf Abteilungen gibt es außerdem 30 Server die Infrastrukturdienste bereitstellen, unter anderem E-Mail und Active Directory. Alle Server verwenden entweder Microsoft Windows Server 2012 oder 2016. Abbildung 4 bietet eine grafische Darstellung der Netzwerktopologie.

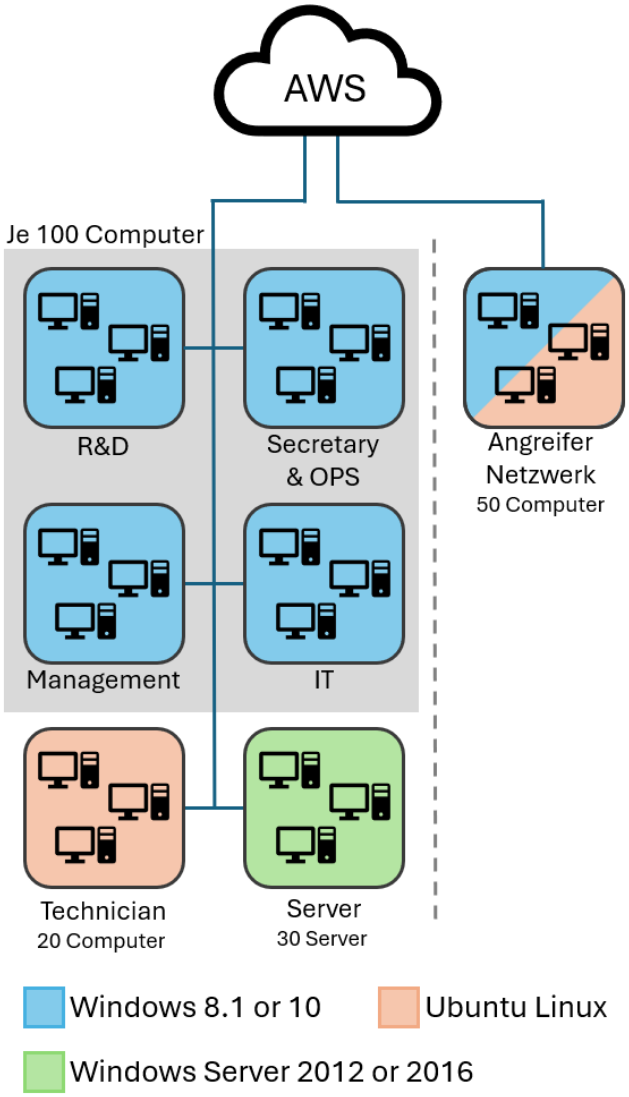

Abbildung 4. Schematische Darstellung der Netzwerk-Topologie. Die Grafik zeigt die Segmentierung in verschiedene Abteilungen sowie die genutzten Betriebssysteme. (Eigene Darstellung in Anlehnung an [3])

Insgesamt deckt der Datensatz die folgenden sieben verschiedenen Angriffsszenarien ab, deren Dauer und eingesetzte Tools in Table I zusammengefasst sind.

Für alle Einträge in dem Datensatz werden mehr als 80 verschiedene Attribute (*Features*) erfasst. Darunter sind „Source IP“, „Timestamp” oder auch “Flow Duration” enthalten.

B. Datenvorverarbeitung

Rohdaten aus Netzwerkverkehr sind für das Training von Machine-Learning-Modellen ohne Vorverarbeitung in der Regel ungeeignet. Sie können unter anderem fehlende Werte oder irrelevante Datenpunkte enthalten, welche die Ergebnisse der Modelle beeinträchtigen können.

Daher wurde eine strukturierte Datenvorverarbeitung (Preprocessing) durchgeführt. Hierfür wurde die Programmierspra-

Tabelle I
ÜBERSICHT DER SIMULIERTEN ANGRIFFE UND VERWENDETEN TOOLS (ÜBERSETZT UND ADAPTIERT NACH [3])

Attacke	**Tool**	**Dauer in Tagen**
Bruteforce	FTP/SSH Patator	1
Denial of Service	• Hulk • Heartleech • GoldenEye • Slowloris • Slowhttptest	1
Web	• Damn Vulnerable Web App (DVWA) • Eigenes Selenium Framework	2
Infiltration	• 1. Level: Dropbox-Download auf einem Windows Computer • 2. Level: Nmap und Portscan	2
Botnet	• Ares (in Python entwickelt): Remote Shell, Datei Upload/Download, Capturing • Screenshots und Key-Logging	1
Distributed Denial of Service und PortScan	Low Orbit Ion Cannon (LOIC) für UDP, TCP oder HTTP-Anfragen	2

che Python mit den Bibliotheken pandas, NumPy und scikit-learn verwendet. Da der Datensatz aus zehn verschiedenen CSV-Dateien besteht, wurden diese zuerst zu einem Dataframe zusammengefasst. Danach wurden alle Leerzeichen in den Spaltennamen entfernt und alle unendlichen Werte durch „NaN“ ersetzt. Zuletzt wurden alle Zeilen entfernt, die einen „NaN“ Wert enthielten.

Da der Datensatz sieben verschiedene Angriffsszenarien enthält, das Ziel dieser Arbeit jedoch ein „Proof of Concept“ zur generellen Anomalieerkennung ist, wurde das Problem auf eine binäre Klassifikation reduziert. Hierfür werden alle mit „Benign“ (Harmlos) markierten Einträge mit einer 0 markiert. Alle restlichen Einträge werden mit einer 1 markiert, da es sich hierbei um Angriffe handelt. Als Resultat entstand ein einheitlicher Datensatz, der den Netzwerkverkehr in lediglich zwei Klassen unterteilt: gutartig (0) oder bösartig (1).

Nun werden Features entfernt, die für das Training von Machine-Learning-Modellen ungeeignet sind. Dazu zählen in diesem Fall vor allem Netzwerk-Identifikatoren wie „Source IP“ oder „Destination IP“. Mit diesen Features ist die Wahrscheinlichkeit hoch, dass die Modelle bestimmte IP-Adressen als gefährlich auswendig lernen. Da Angreifer in realen Szenarien ihre IP-Adressen regelmäßig rotieren oder verschleiern (Spoofing), darf das Modell spezifische IP-Adressen nicht als "bösartig“ auswendig lernen, sondern muss das zugrundeliegende Verhaltensmuster des Datenstroms analysieren. Gleiches gilt für die verwendeten Portnummern. Ebenfalls wird die Spalte „Timestamp“ entfernt, da es sich hier ebenfalls um wenig aussagende Daten handelt, die dazu führen könnten, dass die Modelle bestimmte Uhrzeiten als gefährlich einstufen. Zuletzt wurden Features entfernt, die einzigartige Identifikatoren enthalten, in dem benutzten Datensatz ist das die Spalte „Flow ID“.

Der letzte Schritt der Datenbereinigung umfasste das Entfernen von nicht numerischen Spalten. Diese können die ausgewählten Modelle nicht verarbeiten. Anschließend wurde die Spalte „Label“ (0 oder 1) von den restlichen Features separiert, da sie als Zielvariable für das Training dient. Daraufhin werden die übrigen Daten so skaliert, sodass sie einen Mittelwert von 0 und eine Standardabweichung von 1 haben. Hierfür kam eine *Z-Score-Normalisierung* (Standard Scaler) zum Einsatz. Dabei wird jedes Feature so transformiert, dass es einen Mittelwert (μ) von 0 und eine Standardabweichung (σ) von 1 aufweist. Die mathematische Berechnung für jeden Datenpunkt erfolgt nach der Formel:

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

wobei x der ursprüngliche Wert des Features ist. Dieser Normalisierungsschritt ist zwingend erforderlich, damit Features mit naturgemäß sehr großen numerischen Wertebereichen (wie beispielsweise das insgesamt übertragene Datenvolumen) die Features mit kleinen Wertebereichen während des Trainingsprozesses nicht dominieren.

Zum Abschluss der Datenvorverarbeitung erfolgte der sogenannte *Train-Test-Split*. Um eine objektive Evaluation der trainierten Modelle zu gewährleisten, wurde der bereinigte Datensatz zufällig in zwei disjunkte Mengen unterteilt: 80 % der Daten dienten dem Training der Modelle, während die verbleibenden 20 % ausschließlich für die finale Evaluation (Test-Set) zurückgehalten wurden. Es ist zwingend erforderlich, nicht dieselben Daten für das Training sowie die Evaluation zu nutzen, da das Modell die Trainingsdaten bereits 'gesehen' hat und sich deren Muster gemerkt haben könnte. Würde man dieselben Daten zur Evaluation verwenden, könnte das Modell eine künstlich hohe Genauigkeit erzielen, ohne wirklich neue, unbekannte Fälle korrekt klassifizieren zu können. Die Testdaten simulieren daher reale, ungesehene Daten und ermöglichen somit eine realistische Einschätzung der praktischen Leistungsfähigkeit der Algorithmen.

C. Implementierung und verwendete Technologien

Wie bereits im Abschnitt der Datenvorverarbeitung erwähnt, wurde die gesamte experimentelle Pipeline in der Programmiersprache Python implementiert, welche sich als Standard für Data Science etabliert hat. Die Implementierung stützt sich auf performante Open-Source-Bibliotheken. Der vollständige Quellcode dieser Implementierung ist öffentlich auf GitHub zugänglich [1].

Ergänzend zu den bereits genannten Bibliotheken Pandas und NumPy, die für die Datenvorverarbeitung genutzt wurden, erfolgte das Herunterladen des Datensatzes automatisiert über Kagglehub. Die Speicherung des Dataframes auf dem Datenträger wurde mittels Joblib realisiert. Letzteres erlaubt es, die vorverarbeiteten Daten binär abzuspeichern, was die Ladezeiten für das spätere Modelltraining erheblich reduziert.

Für das Training der klassischen Machine-Learning-Algorithmen (Decision Tree, Random Forest und Logistische Regression) wurde das Scikit-Learn-Framework verwendet, welches hochoptimierte, CPU-basierte Implementierungen dieser Modelle bietet. Das Gradient-Boosting-Verfahren wurde hingegen mit der nativen XGBoost-Bibliothek realisiert, da diese effiziente und parallelisierte Algorithmen für baumbasierte Modelle bereitstellt.

Das künstliche Neuronale Netz wurde mit dem Deep-Learning-Framework PyTorch erstellt, da diese Bibliothek es ermöglicht die Berechnungen parallel auf einer Grafikkarte (mittels *CUDA* [12]) durchzuführen. Das ist für Neuronale Netze, insbesondere bei der Größe des Datensatzes, sehr wichtig, um die Trainingszeit in einem angemessenen Rahmen zu halten.

Die Visualisierung der Ergebnisse, wie die *Konfusionsmatrizen*, erfolgte mit den Bibliotheken Seaborn und Matplotlib.

Um die gemessenen Trainingszeiten der Modelle später einordnen zu können, wird nachfolgend die verwendete Hardware spezifiziert: Aufgrund des hohen Arbeitsspeicherbedarfs des unkomprimierten Gesamtdatensatzes fand die Datenvorverarbeitung auf einem Server mit 64 GB RAM statt. Das anschließende Modelltraining sowie die Evaluation erfolgten lokal auf einem System mit einem Intel Core Ultra 7 265K Prozessor und 48 GB RAM. Für das hardwarebeschleunigte Training des Neuronalen Netzes (via CUDA) wurde eine NVIDIA GeForce RTX 5080 verwendet.

IV. Evaluation und Ergebnisse

In diesem Kapitel werden die Ergebnisse der trainierten Machine-Learning-Modelle präsentiert und verglichen. Der Fokus liegt dabei auf der Erkennungsrate sowie der Trainingsdauer.

A. Bewertungsmetriken

Für Anomalieerkennungssysteme ist es nicht ausreichend, nur die *Genauigkeit* (Accuracy) der Modelle zu betrachten, da Netzwerkverkehr nie ausgeglichen ist. Der Datensatz hat ein Verhältnis von etwa 13:1 von harmlosem Verkehr zu Angriffen. Das führt dazu, dass ein Modell, das ausschließlich harmlosen Verkehr (Benign) vorhersagt, auch eine hohe Genauigkeit (über 90 %) hat, ohne einen einzigen Angriff erkannt zu haben. Deshalb werden stattdessen andere Metriken für den Vergleich der Modelle verwendet. Diese basieren auf den Werten einer sogenannten *Konfusionsmatrix* (Confusion Matrix). Diese Matrix ist eine visuelle Darstellung zur Leistungsbewertung eines Models. Dabei werden die vom Modell vorhergesagten Fälle den tatsächlichen Fällen gegenübergestellt. Im Falle einer binären Klassifikation ergibt sich daraus eine 2×2-Matrix mit vier Werten:

- **True Positives (TP)**
  Bösartige Angriffe, die vom Modell korrekterweise als Angriff erkannt wurden.
- **True Negatives (TN)**
  Harmloser Netzwerkverkehr, der korrekterweise als harmlos eingestuft wurde.
- **False Positives (FP)**
  Harmloser Verkehr, der fälschlicherweise als Angriff gemeldet wurde (Fehlalarm).
- **False Negatives (FN)**
  Tatsächliche Angriffe, die vom System übersehen und fälschlicherweise als harmlos eingestuft wurden.

Folgende Metriken werden zusätzlich betrachtet:

- **Precision** [13]
  Die Precision misst wie viele der vom Modell als positiv vorhergesagten Fälle auch tatsächlich positiv sind. Eine hohe Precision bedeutet wenige Fehlalarme, was die sogenannte Alert Fatigue bei Systemadministratoren verhindert. Die Precision wird wie folgt berechnet:
  $$Precision = \frac{TP}{TP + FP} \tag{2}$$
- **Recall** [14]
  Der Recall misst, wie viele der tatsächlich positiven Fälle das Modell korrekt gefunden hat. Ein hoher Recall ist besonders wichtig, wenn das Übersehen eines positiven Falls kritisch ist, was für KRITIS von großer Bedeutung ist. Der Recall wird wie folgt berechnet:
  $$Recall = \frac{TP}{TP + FN} \tag{3}$$

Erhöht man die Precision, damit weniger Fehlalarme verursacht werden, sinkt meist der Recall, was zu mehr übersehenen Fällen führt. Durch dieses Spannungsverhältnis von Precision und Recall wird auch der F1-Score der Modelle verglichen:

- **F1-Score** [14]
  Der F1-Score ist das harmonische Mittel aus Precision und Recall. Der Wert liegt zwischen 0, was ein schlechtes Ergebnis bedeutet, und 1, was ein perfektes Ergebnis bedeutet. Er liefert einen ausgewogenen Indikator für die Gesamtleistung eines Modells bei unbalancierten Daten. Es wird das harmonische Mittel verwendet, da es große Unterschiede zwischen Precision und Recall stärker bestraft als das arithmetische Mittel. Ein Modell mit

[1] Quellcode und Skripte sind abrufbar unter: https://github.com/basimbennaji/KI-basierte-Angriffserkennung-in-kritischen-Infrastrukturen

hoher Precision und niedrigem Recall-Wert würde sonst einen mittleren F1-Score erhalten, wobei das Modell wenig Fehlalarme verursachen, aber viele Angriffe nicht erkennen würde. Der F1-Score wird wie folgt berechnet:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{4}$$

B. Trainingszeiten und Ressourceneffizienz

Ein wichtiger Aspekt für den praktischen Einsatz von den getesteten Machine-Learning-Modellen in KRITIS ist die Skalierbarkeit und die nötige Rechenleistung. Wenn die Modelle schnell genug trainiert werden können, ist es möglich die Modelle immer wieder auf neuen Daten zu trainieren und damit diese auch verbessern.

Die Trainingsdauer der Modelle auf dem Datensatz zeigen bereits signifikante Unterschiede. Die baumbasierten Modelle ließen sich am schnellsten trainieren. Der Decision Tree benötigte für das Training auf über 6 Millionen Datensätzen nur 56,62 Sekunden. Der Random Forest hat mit einer Trainingsdauer von 61,08 Sekunden nur etwas länger gebraucht. Das Neuronale Netz hat mit 113,5 Sekunden etwa doppelt so lange gebraucht, jedoch hat es stark von der hardwareseitigen Beschleunigung mittel CUDA profitiert.

Die Logistische Regression scheiterte an den Trainingsdaten. Nach 5921,1 Sekunden Trainingsdauer konnte der Optimierungsalgorithmus nicht konvergieren. Das Programm wurde mit einer ConvergenceWarning beendet.

Besonders schnell erwies sich das Gradient-Boosting-Modell. Mit einer Trainingsdauer von nur 11,17 Sekunden im ersten Durchlauf ist es mit Abstand das schnellste Modell in diesem Experiment. Hier wird auch deutlich, wieso dieses Modell häufig für tabellarische Daten verwendet wird. Durch die Verwendung von *OpenMP* können viele CPU-Kerne beansprucht werden, um so die Trainingsdauer der einzelnen Bäume zu verringern. Dies ist essenziell, da die Bäume beim Boosting-Verfahren architekturbedingt weiterhin sequenziell auf die Fehler der jeweils vorherigen Bäume trainiert werden müssen. Da XGBoost zudem die Nutzung einer GPU unterstützt, kann die Trainingsdauer weiter reduziert werden. Durch die GPU-Beschleunigung konnte eine Trainingsdauer von nur 4,49 Sekunden erreicht werden. Da die Eingabedaten jedoch weiterhin im Arbeitsspeicher (RAM) lagen und nicht direkt im Grafikspeicher (VRAM), gab es noch einen Konvertierungs- und Transport-Overhead. Wurden die Eingabedaten hingegen vorab in den VRAM übertragen, sodass dieser Overhead entfällt, ergab sich eine Trainingsdauer von 1,81 Sekunden.

C. Klassifikationsergebnisse

Die Evaluierung auf dem zurückgehaltenen Test-Datensatz (1.577.859 Einträge) zeigte, dass die baumbasierten Modelle den komplexeren Modellansätzen bei diesen tabellarischen Netzwerkverkehrsdaten deutlich überlegen sind. Eine detaillierte Gegenüberstellung aller relevanten Metriken sind in den Tabellen II und III zusammengefasst.

Tabelle II
KLASSIFIKATIONSMETRIKEN DER TRAINIERTEN MODELLE

Modell	**Precision**	**Recall**	**F1-Score**
Decision Tree	99,987 %	99,988 %	99,987 %
Random Forest	99,935 %	99,979 %	99,957 %
Neuronales Netz	98,824 %	99,946 %	99,382 %
Logistische Regression	87,778 %	99,134 %	93,111 %
XGBoost (CPU)	99,977 %	99,999 %	99,988 %
XGBoost (GPU)	99,977 %	99,999 %	99,988 %
XGBoost (GPU+VRAM)	99,977 %	99,999 %	99,988 %

Tabelle III
KONFUSIONSMATRIXWERTE DER TRAINIERTEN MODELLE

Modell	**TP**	**TN**	**FP**	**FN**
Decision Tree	115.224	1.462.606	15	14
Random Forest	115.214	1.462.546	75	24
Neuronales Netz	115.176	1.461.250	1.371	62
Logistische Regression	114.240	1.446.715	15.906	998
XGBoost (CPU)	115.237	1.462.594	27	1
XGBoost (GPU)	115.237	1.462.599	22	1
XGBoost (GPU+VRAM)	115.235	1.462.599	22	3

Der Decision Tree erzielte hervorragende Gesamtergebnisse. Mit einer Precision von 99,987 % und einem Recall von 99,988 % ordnete er die Daten nahezu perfekt ein. Bei den tatsächlichen 115.238 Angriffen im Test-Set übersah das Modell nur 14 Angriffe (False Negatives) und erzeugte nur 15 Fehlalarme (False Positives) bei über 1,46 Millionen harmlosen (benigne) Netzwerkverbindungen.

Der Random Forest zeigte eine leicht schlechtere Leistung als der Decision Tree. Mit einer Precision von 99,935 % und einem Recall von 99,979 % übersah das Modell 24 Angriffe und löste 75 Fehlalarme aus.

Das Neuronale Netz erzielte zwar einen guten Recall von 99,946 % und übersah 62 Angriffe, offenbarte jedoch mit 98,824 % eine etwas schwächere Precision. Das führte dazu, dass 1.371 Fehlalarme verursacht wurden. Eine derart hohe Fehlalarmquote würde die IT-Abteilung eines Unternehmens stark belasten.

Die Logistische Regression schnitt mit Abstand am schlechtesten ab. Mit einer Precision von nur 87,778 % und einem Recall von 99,134 % generierte das Modell 15.906 Fehlalarme und übersah gleichzeitig 998 Angriffe. Dies belegt, dass einfache lineare Modelle für die komplexe Netzwerkangriffserkennung ungeeignet sind.

Das XGBoost-Modell lieferte über alle Hardware-Konfigurationen hinweg die besten Klassifikationsergebnisse. In der Standardkonfiguration (nur CPU) übersah das Modell lediglich einen einzigen Angriff (FN = 1) und produzierte nur 27 Fehlalarme. Bemerkenswert ist die minimale Varianz bei der hardwarebeschleunigten Ausführung:

Durch die abweichende Fließkomma-Arithmetik (Floating-Point Math) der Grafikkarte sank die Anzahl der False Positives bei der GPU-Ausführung auf 22, während bei der Speicheroptimierung (direktes Laden in den VRAM) 3 Angriffe übersehen wurden. Dennoch klassifizierte XGBoost in allen Varianten mit einem F1-Score von 99,988 % nahezu fehlerfrei. Dies bestätigt die Annahme, dass Ensemble-Methoden [10] für komplexe, tabellarische Netzwerkdaten die beste Option darstellen.

Um das Ausmaß der Leistungsdifferenz zu verdeutlichen, beschränkt sich Abbildung 5 ausschließlich auf die Gegenüberstellung der Fehlklassifikationen (FP und FN) des hardwarebeschleunigten XGBoost-Modells (ohne VRAM-Vorabladen) und der Logistischen Regression. Es wurde eine logarithmische Darstellung verwendet, um die massiven Größenordnungsunterschiede passend abzubilden.

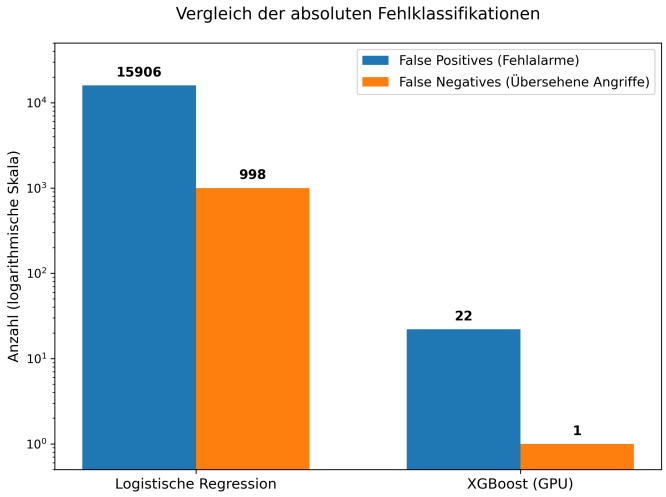

Abbildung 5. Gegenüberstellung der absoluten Fehlklassifikationen zwischen dem schwächsten Modell (Logistische Regression) und dem leistungsstärksten Modell (XGBoost). Um die massiven Leistungsdifferenzen darzustellen, wurde eine logarithmische Skalierung für die Y-Achse gewählt.

V. Diskussion

Dieses Kapitel interpretiert die experimentellen Ergebnisse im Hinblick auf die eingangs definierten Fragen. Es wird diskutiert, welche algorithmischen Ansätze sich am besten eignen und inwiefern ein operativer Einsatz in kritischen Infrastrukturen realistisch ist, wobei der Fokus auf der technologischen Machbarkeit und dem reinen Modellvergleich liegt, und nicht auf einem final einsatzbereiten Produkt.

Die vorliegende Evaluierung hat gezeigt, dass die Wahl des zugrunde liegenden Machine-Learning-Algorithmus einen massiven Einfluss auf die Praxistauglichkeit eines Intrusion Detection Systems (IDS) in kritischen Infrastrukturen hat. Die Ergebnisse lassen sich im Wesentlichen in drei zentrale Erkenntnisse unterteilen: die Überlegenheit baumbasierter Modelle bei tabellarischen Daten, die kritische Bedeutung von False Positives im operativen Betrieb sowie die Notwendigkeit von ressourceneffizientem Training.

A. *Modellvergleich*

Es wurde empirisch belegt, dass komplexe Netzwerkangriffe stark nicht-lineare Muster in den Verbindungsdaten hinterlassen. Das vollständige Versagen der Logistischen Regression unterstreicht, dass einfache lineare Trennfunktionen nicht in der Lage sind, diese Muster zu erfassen. Jedoch konnte auch das mathematisch komplexe Neuronale Netz die baumbasierten Ensemble-Modelle wie Random Forest oder XGBoost nicht übertreffen.

Dies deckt sich mit den aktuellen Erkenntnissen der internationalen Data-Science-Forschung: Wie Grinsztajn *et al.* in ihrer umfangreichen Benchmark-Studie auf der NeurIPS-Konferenz empirisch belegen konnten, dominiert Deep Learning zwar bei unstrukturierten Daten, bei strukturierten, tabellarischen Datensätzen erzielen baumbasierte Algorithmen jedoch durchgehend überlegene Ergebnisse [15]. Dies liegt unter anderem daran, dass sie durch ihre hierarchische Knotenstruktur natürliche Schwellenwerte für Netzwerk-Features (wie Paketgrößen oder Portnummern) bilden können.

B. *Alert Fatigue*

Betrachtet man die Precision oder den Recall isoliert, lieferten fast alle Modelle – mit Ausnahme der Logistischen Regression – hervorragende Werte. Für die IT-Sicherheitsabteilung (bzw. das Security Operations Center) eines Unternehmens ist jedoch die absolute Anzahl der Fehlalarme (False Positives) die wichtigste Metrik. Jeder Fehlalarm bindet wertvolle Arbeitszeit von Sicherheitsanalysten.

Das Neuronale Netz produzierte im Testlauf 1.371 Fehlalarme. In einem realen KRITIS-Netzwerk mit hunderten Millionen Verbindungen pro Tag würde diese Fehlerquote zu tausenden falschen Warnungen führen, was in einer sogenannten „Alert Fatigue“ (Alarmmüdigkeit) resultiert. In diesem Zustand ständiger Überreizung werden selbst kritische Warnungen von den Analysten häufig übersehen oder ignoriert, da das grundlegende Vertrauen in die Zuverlässigkeit des Erkennungssystems verloren gegangen ist [5].

Im drastischen Gegensatz dazu bewies XGBoost mit lediglich 22 Fehlalarmen und nur einem einzigen übersehenen Angriff eine herausragende Präzision, die einen effizienten und fokussierten Betrieb von IT-Sicherheitsabteilungen überhaupt erst ermöglicht.

C. *Ressourceneffizienz und kontinuierliches Lernen*

Ein modernes IDS muss nicht nur präzise sein, sondern sich auch schnell an neue Bedrohungslagen (Zero-Day-Exploits) anpassen können. Die gemessenen Trainingszeiten offenbarten hier die größte Stärke von XGBoost. Durch die native Unterstützung von Hardwarebeschleunigung konnte das Modell bei direkter VRAM-Nutzung in unter zwei Sekunden auf über 6 Millionen Datensätzen trainiert werden. Diese extreme Effizienz ermöglicht ein ressourcenschonendes, nahezu echtzeitfähiges „Continuous Learning“, bei dem das Modell im laufenden Betrieb kontinuierlich mit neuen Netzwerkdaten nachtrainiert wird, ohne dass massive Rechenzentren oder teure Cloud-Instanzen gemietet werden müssen.

VI. Fazit

Die vorliegende Arbeit untersuchte verschiedene Machine-Learning-Modelle für die netzwerkbasierte Anomalieerkennung in kritischen Infrastrukturen. Angesichts der sehr großen Datenmengen und der Notwendigkeit höchster Präzision im IT-Sicherheitsumfeld erwiesen sich baumbasierte Ensemble-Methoden, insbesondere der XGBoost-Algorithmus, als die mit Abstand vielversprechendste Lösung für die Klassifikation tabellarischer Netzwerkdaten.

Während lineare Modelle wie die Logistische Regression an der Nicht-Linearität der Angriffsmuster scheiterten und klassische Deep-Learning-Ansätze wie Neuronale Netze durch zu hohe Fehlalarmraten das Risiko der *Alert Fatigue* erhöhten, lieferte XGBoost eine nahezu fehlerfreie Trennschärfe. Mit lediglich 22 Fehlalarmen (False Positives) bei über 1,46 Millionen gutartigen Verbindungen erfüllt das Modell die strengen operativen Anforderungen eines modernen Security Operations Centers (SOC).

Des Weiteren zeigte das Modell eine sehr ausgeprägte Ressourceneffizienz. Durch die gezielte Hardwarebeschleunigung mittels CUDA und die Optimierung der Speicherarchitektur durch das Vorabladen in den VRAM konnte die Trainingszeit für über 6 Millionen Datensätze auf unter zwei Sekunden reduziert werden. Diese technologische Basis ermöglicht ein ressourcenschonendes, kontinuierliches Nachtrainieren im laufenden Betrieb.

Für zukünftige Forschungsarbeiten ergeben sich aus diesen Erkenntnissen vielversprechende Anknüpfungspunkte. Ein logischer nächster Schritt wäre die Implementierung des trainierten XGBoost-Modells in eine Echtzeit-Datenpipeline (beispielsweise mit Apache Kafka), um die Latenzen und den Ressourcenverbrauch unter realen Live-Bedingungen mit echten Daten zu validieren.

Darüber hinaus sollte untersucht werden, wie robust das baumbasierte Modell gegenüber gezielten Angriffen ist. Hierbei versuchen Angreifer, die Entscheidungsstrukturen des Modells durch minimal und gezielt manipulierte Netzwerkpakete zu täuschen. Abschließend sollte die Automatisierung des Trainingsprozesses evaluiert werden. Mit Ansätzen aus dem Bereich AutoML könnte das Modell ohne manuelle Eingriffe kontinuierlich mit neuen Daten trainiert und an eine sich wandelnde Bedrohungslandschaft angepasst werden [16].

Erklärung zur Nutzung von KI-Werkzeugen

Gemäß der KI-Richtlinie der Arbeitsgruppe Computer Systems and Telematics wird im Folgenden der Einsatz von KI-Werkzeugen in dieser Arbeit dokumentiert.

Verwendete Tools:

- Google Gemini 3 Fast und 3.1 Pro (Zuletzt verwendet im März 2026)
- OpenAI ChatGPT 5 (Zuletzt verwendet im Dezember 2025)
- Perplexity Pro mit der automatischen Modellauswahl sowie Claude Sonnet 4.6 und Sonar (Zuletzt verwendet im März 2026)

Einsatzzwecke:

- **Code-Generierung:**
  Die Tools Gemini 3.1 Pro und Claude Sonnet 4.6 wurden intensiv zur Generierung von Code-Snippets für die Datenvorverarbeitung (Preprocessing) und die Implementierung der Machine-Learning-Modelle sowie für die Erstellung der Visualisierungsskripte (Plots) verwendet.
- **Ideen für das Experiment:**
  ChatGPT 5 wurde für die Ideenfindung für das Experiment verwendet. Es wurde auch diskutiert welche Vergleichsparameter am besten geeignet sind.
- **Literaturrecherche:**
  Perplexity Pro wurde bei der Suche nach geeigneten wissenschaftlichen Quellen (z.B. Recherche nach aktuellen Papern zum CSE-CIC-IDS2018 Datensatz) verwendet.
- **Sprachliche Überarbeitung und Strukturierung:**
  Gemini 3 Fast und 3.1 Pro wurden als Assistenz zur Gliederung von Absätzen, zur sprachlichen Optimierung und für Formulierungsvorschläge genutzt. Die Kernleistung (Analyse, Argumentation und finale Ausformulierung) wurde eigenständig erbracht.
- **LaTeX Formatierung:**
  Gemini 3 Fast, 3.1 Pro und Claude Sonnet 4.6 wurden für die Formatierung vom LaTeX Dokument und für die Einhaltung der IEEE-Standards verwendet.
- *Hinweis: Die Nutzung der KI zur Klärung von allgemeinen Verständnisfragen erfolgte gemäß der Richtlinie als persönliche Lernhilfe und ist hier nur der Vollständigkeit halber erwähnt.*

Exemplarisches Nutzungsprotokoll (Prompts):

- *Beispiel Code-Generierung:* „Schreibe ein Python-Skript, das den CSE-CIC-IDS2018 Datensatz lädt, alle CSV-Dateien zusammenfasst und die Daten bereinigt.“
  Antwort:
  3. Step 1: Data Cleaning Preprocessing Script
  You need to handle Ïnfinityänd "NaN"values which are common in this dataset (e.g., when dividing by zero duration).
  Save this as preprocessing.py:
  import pandas as pd
  import numpy as np
  from sklearn.model_selection import train_test_split
  from sklearn.preprocessing import StandardScaler, LabelEncoder
  def load_and_clean_data(filepath, sample_size=None):
  print(f"Loading data from filepath...")
  [...]
  Split into Train/Test
  X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
  return X_train, X_test, y_train, y_test, len(le.classes_)
- *Beispiel Literaturrecherche:* „i want you to find out what kind of intrusion detection system critical infrastructure uses nowadays. who uses signature based systems and

who uses anomaly based systems. i need references“
Antwort:
Many critical infrastructure operators use both signature-based and anomaly-based IDS today, often as combined or layered systems; purely one or the other is less common in modern deployments.
[...]
Who uses signature-based systems
Signature-based IDS are still the default for:
Perimeter network IDS/IPS in IT segments of critical infrastructure
[...]

- *Beispiel Textarbeit:* „Das ist mein Text für Absatz XY: [...] Nenne mir alle Rechtschreib- und Grammatikfehler mit Zeilenangabe und Verbesserungsvorschlag. Nenne auch Verbesserungsvorschläge für die Formulierung.“
  Antwort:
  [...]
  Es gibt nur ein paar winzige Tippfehler (z. B. "hervoragende") und zwei, drei Stellen, an denen wir die umgangssprachlichen Formulierungen noch einen kleinen Tick akademischer machen können.
  Hier sind meine konkreten Verbesserungsvorschläge (mit Begründung), gefolgt von dem fertigen, polierten Text zum Kopieren:
  Die konkreten Verbesserungen:
  Absatz 1:
  Ist: „...ist in Table `\ref{tab:ergebnisse}` zusammengefasst.“
  Verbesserung: Wenn du deine Arbeit auf Deutsch schreibst, solltest du „Tabelle `\ref{...}`“ (und später „Abbildung `\ref{...}`“) schreiben, damit Sprache und Layout konsistent bleiben.
- *Beispiel Debugging:* „Traceback (most recent call last):
  File "d:_forest_random_forest.py", line 4, in <module>
  from visualization_utils import evaluate_and_plot
  ModuleNotFoundError: No module named ’visualization_utils’.“
  [...]
  Antwort:
  This is a classic Python import error. Because you moved the model into a subfolder (random_forest/), it can no longer ßee"the visualization_utils.py file in the main folder.
  You need to tell Python to look in the parent directory (../) for that file.
  Here is the fix. You need to add this code block to the very top of model_random_forest.py (and all other model files), before the other imports.
  [...]
- *Beispiel LaTeX:* „Schreibe für diese Quelle ein BibTeX Eintrag: [Infos der Quelle].“
  Antwort:
  [...]
  @articlewicaksana2024comparative,
  title=Comparative Analysis of Random Forest and Xgboost Performance for Network Flow-Based Malware Classification,
  author=Wicaksana, Fajar Adji and others,
  journal=INOVTEK Polbeng - Seri Informatika,
  year=2024
  [...]
- *Beispiel LaTeX:* „wo sollte ich TP,FP und FN erklären? in fußnoten?“
  Antwort:
  Das ist eine sehr gute formale Frage! Mein klarer Rat für ein IEEE-Paper: Bitte nutze dafür keine Fußnoten!
  In den Geisteswissenschaften sind Fußnoten für Erklärungen super, aber in der Informatik und im Machine Learning sind True Positives, False Positives etc. das absolute Herzstück deiner Ergebnisse. Wenn [...]

LITERATUR

[1] IBM. Was ist ein Zero-Day-Exploit? Zuletzt abgerufen am 12. März 2026. [Online]. Available: https://www.ibm.com/de-de/think/topics/zero-day

[2] Bundesamt für Sicherheit in der Informationstechnik. (2024) Orientierungshilfe zum einsatz von systemen zur angriffserkennung. Zuletzt abgerufen am 12. März 2026. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/KRITIS/oh-sza.pdf?__blob=publicationFile&v=18

[3] University of New Brunswick. CSE-CIC-IDS2018 on AWS. Zuletzt abgerufen am 12. März 2026. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html

[4] F. Wicaksana and C. Umam, “Comparative analysis of random forest and xgboost performance for network flow-based malware classification,” *INOVTEK Polbeng - Seri Informatika*, vol. 11, pp. 108–115, 02 2026.

[5] T. Ban, N. Samuel, T. Takahashi, and D. Inoue, “Combat security alert fatigue with ai-assisted techniques,” in *Proceedings of the 14th Cyber Security Experimentation and Test Workshop*, ser. CSET ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 9–16, zuletzt abgerufen am 14. März 2026. [Online]. Available: https://doi.org/10.1145/3474718.3474723

[6] M. Zamani and M. Movahedi, “Machine learning techniques for intrusion detection,” 2015, zuletzt abgerufen am 14. März 2026. [Online]. Available: https://arxiv.org/abs/1312.2177

[7] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” 01 2010, pp. 305–316, zuletzt abgerufen am 14. März 2026.

[8] Palo Alto Networks. Was ist ein Intrusion-Detection-System? Zuletzt abgerufen am 12. März 2026. [Online]. Available: https://www.paloaltonetworks.de/cyberpedia/what-is-an-intrusion-detection-system-ids

[9] Proofpoint. Was ist ein Intrusion Detection System (IDS)? Zuletzt abgerufen am 12. März 2026. [Online]. Available: https://www.proofpoint.com/de/threat-reference/intrusion-detection-system-ids

[10] Jacob Murel Ph.D., Eda Kavlakoglu - IBM. Was ist ensemble-lernen? Zuletzt abgerufen am 14. März 2026. [Online]. Available: https://www.ibm.com/de-de/think/topics/ensemble-learning

[11] IBM. Was ist XGBoost? Zuletzt abgerufen am 13. März 2026. [Online]. Available: https://www.ibm.com/de-de/think/topics/xgboost

[12] Fred Oh - Nvidia. (2012) What is cuda? Zuletzt abgerufen am 13. März 2026. [Online]. Available: https://blogs.nvidia.com/blog/what-is-cuda-2/

[13] Google. (2026) Klassifizierung: Genauigkeit, trefferquote, präzision und zugehörige messwerte. Zuletzt abgerufen am 13. März 2026. [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=de

[14] Niklas Lang - Data Base Camp. (2023) F1-score – einfach erklärt! Zuletzt abgerufen am 13. März 2026. [Online]. Available: https://databasecamp.de/statistik/f1-score

[15] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc.,

2022, pp. 507–520, zuletzt abgerufen am 14. März 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/0378c7692da36807bdec87ab043cdadc-Paper-Datasets_and_Benchmarks.pdf

[16] Microsoft. (2025) Was ist automatisiertes maschinelles lernen (automl)? Zuletzt abgerufen am 14. März 2026. [Online]. Available: https://learn.microsoft.com/de-de/azure/machine-learning/concept-automated-ml?view=azureml-api-2